

블록체인 nonce 관리 사례와 권장 아키텍처

핵심 요약

공개 자료를 종합하면, 성숙한 운영자들의 공통점은 “**송신자별 단일 권한자(single writer) + 오프체인 권위 상태(authoritative pending state) + 온체인 재조정(reconciliation)**” 입니다. MetaMask는 로컬 confirmed/pending 내역과 네트워크 nonce를 합쳐 계산하고 `releaseLock` 기반 잠금을 둡니다. Kaia는 Redis·Postgres 같은 오프체인 저장소와 중앙화 nonce manager를 공식 권고합니다. OpenZeppelin Relayer는 nonce를 원자적으로 할당해 큐에 넣고, 자동 재제출과 교체까지 수행합니다. Fireblocks도 EVM에서 고급 nonce 관리와 stuck tx 감시를 제공하며, 실패했지만 채굴되지 않은 거래의 nonce 재사용을 지원합니다. ¹

대량 처리에서는 EOA의 순차 nonce만으로는 병목이 생기므로, 선도 업체들은 **여러 송신 지갑으로 샤딩, 스마트 계정 배치 실행, ERC-4337 keyed nonce** 같은 우회 수단을 함께 씁니다. Alchemy는 스마트 월렛에서 `nonceOverride` / `nonceKey` 로 병렬 전송을 지원하고, ERC-4337 EntryPoint의 `NonceManager` 는 계정별로 `key` 와 `sequence` 를 분리해 관리합니다. Coinbase는 CDP Wallets/Smart Wallet/Bundler가 nonce 관리와 fee sequencing을 맡는다고 명시하고, Fireblocks는 복수 출금 지갑의 round-robin 운영을 권합니다. ²

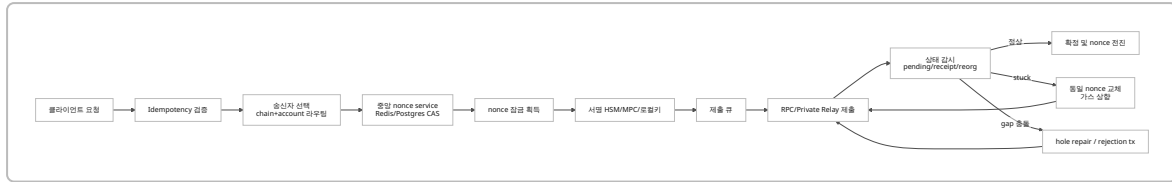
체인마다 관리 대상도 다릅니다. EVM은 계정 nonce가 핵심이고, Bitcoin은 계정 nonce가 없으므로 **UTXO 예약·입력 충돌-RBF/CPFP** 가 핵심입니다. Solana는 recent blockhash 만료와 durable nonce account, Algorand는 `FirstValid/LastValid` 와 `Lease` , Starknet과 Cosmos는 계정 sequence/nonce를 관리해야 합니다. 따라서 아키텍처 키는 단순히 `(account)` 가 아니라 보통 `(chain, sender, nonce-domain)` 이어야 합니다. ³

사례 비교

업체	공개된 방식	nonce 관련 공개 포인트	시사점	근거
MetaMask	클라이언트 로컬 tracker	<code>@metamask/nonce-tracker</code> 는 로컬 confirmed 최고 nonce와 <code>eth_getTransactionCount</code> 결과의 최대값을 기준으로 <code>nextNonce</code> 를 만들고, pending tx 반영 전까지 <code>releaseLock</code> 을 유지	로컬 pending state + mutex 패턴의 대표 사례	⁴
Coinbase CDP	관리형 지갑/스마트 월렛	EVM <code>sendTransaction</code> 이 gas estimation·nonce management·signing·broadcast를 자동 처리. Smart Wallet/Bundler는 Shopify 사례에서 nonce management·fee sequencing을 제공	관리형 nonce 서비스 와 4337 bundler형 운영	⁵
Binance Wallet	지갑 내부 자동화	현재는 “confirmed nonce + 1”이 아니라 pending mempool 기준 + 1 을 사용하며, 사용자가 nonce를 신경 쓰지 않도록 자동 생성	단순 <code>latest+1</code> 보다 pending-aware 가 실무적으로 안전	⁶

업체	공개된 방식	nonce 관련 공개 포인트	시사점	근거
Infura	원시 RPC + 관찰 도구	<code>eth_getTransactionCount</code> , <code>newPendingTransactions</code> 구독, raw tx용 MEV 보호를 제공하지만, 공개 문서상 별도 nonce allocator는 보이지 않음	인프라 빌딩블록 제공자에 가깝고, nonce orchestration은 애플리케이션 몫	7
Alchemy	두 갈래 모델	Wallet APIs는 retries·status tracking·batching을 관리하지만, Reinforced Transactions는 “nonce assignment는 관리하지 않는다”고 명시. 스마트월렛은 <code>nonceKey</code> 로 병렬 전송 지원	관리형 API와 raw RPC를 분리; raw path에서는 자체 nonce 서비스 필요	8
Safe	온체인 스마트계정 + 오프체인 서비스	Safe contract는 tx nonce를 사용하고, Transaction Service는 오프체인 서명 수집·인덱싱·reorg 처리·DB/RabbitMQ/Celery 구조를 공개. <code>createRejectionTransaction</code> 으로 특정 nonce 무효화 가능	멀티시그/스마트계정은 EOA nonce 대신 계약 nonce + 서명 수집 서비스 가 핵심	9
Argent Ready	스마트월렛 + guardian/co-signer	Web Wallet은 backend guardian이 공동 서명자이며, 스마트 계정 구조를 사용. Starknet nonce는 계정 계약 단위로 순차 증가	키·서명 모델이 중심 이고, nonce는 체인/계약 계층에서 관리	10
Fireblocks	MPC 기반 관리형 출금 시스템	고급 EVM nonce 관리, stuck tx 감시, 실패·미채굴 tx nonce 재사용 지원. 복수 출금 지갑 round-robin과 active/active 서명 컴포넌트, HSM/MPC 권고	거래소·커스터디에 가까운 운영형 패턴 공개 수준이 높음	11
BitGo	HSM/MPC + recovery API	BitGo는 unsigned tx에 nonce를 추가한 뒤 HSM으로 보낸다고 설명. 또 “Resolve Nonce Holes” 가이드를 별도로 제공하고 stuckTx webhook도 제공	gap 탐지·복구 API 를 노출한 드문 사례	12
Kaia	체인 문서가 직접 운영 패턴 권고	Redis/Postgres 기반 오프체인 nonce store, 중앙 nonce manager, 같은 nonce+더 높은 fee 재시도, 다중 계정 병렬화, fee delegation에도 동일 규칙 권고	한국계 체인 문서 중 가장 실무적 가이드	13
Lambda256 Nodit	데이터/노드 인프라	“Get Next Nonce by Account” API를 제공하며, exchange-grade trusted node·high data consistency·chain-specific process를 강조	한국 인프라 업체는 정확한 상태조회와 체인별 데이터 처리 를 전면에 둠	14
국내 거래소 공개 문서	공개 범위 제한적	Upbit·Korbit·Bithumb의 공개 API는 주문·입출금·인증 위주이며, 내부 hot-wallet nonce queue/allocator 구조는 공개되지 않음	거래소는 대체로 nonce 엔진을 비공개 운영	15

공개 사례는 크게 두 부류로 갈립니다. **원시 RPC/노드 제공형**은 nonce 계산의 재료만 제공하고, **관리형 relay/custody/smart-wallet형**은 nonce를 서비스 책임으로 끌어옵니다. 실무적으로는 후자가 운영 장애를 줄이지만, 전자는 유연성과 비용 통제가 좋습니다. 16



위 흐름은 Kaia의 중앙 nonce manager, MetaMask의 lock, OpenZeppelin Relay의 원자적 할당·큐, BitGo의 hole repair, Alchemy의 retry 흐름을 종합한 실무형 모델입니다. 17

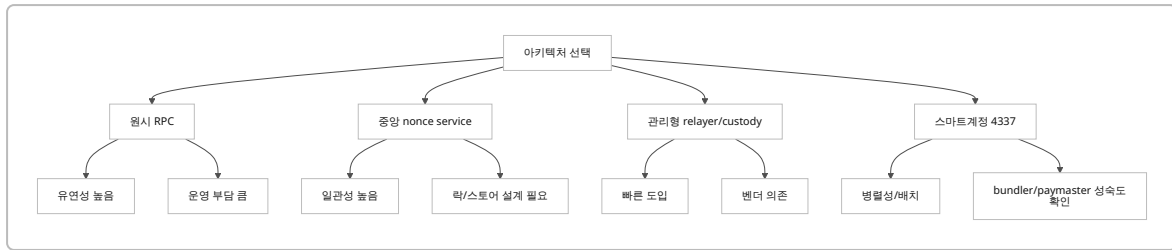
기술 패턴 분석

핵심은 **온체인 상태만으로는 부족하고, 오프체인 pending 상태가 반드시 필요하다**는 점입니다. EVM의 `eth_getTransactionCount` 는 네트워크 기준 “다음 nonce” 를 알려주지만, 고속 병렬 전송에서는 서비스 내부에 이미 할당했으나 아직 네트워크가 완전히 반영하지 못한 pending 의도가 생깁니다. 그래서 MetaMask는 로컬 tx history를 섞고, Kaia는 오프체인 store를 권고하며, OpenZeppelin은 서버 측에서 nonce를 원자적으로 할당한 뒤 큐잉합니다. 18

갭 처리와 재시도도 중요합니다. Kaia는 gap이 생기면 뒤 nonce가 모두 멈춘다고 설명하고, stuck tx는 **같은 nonce에 더 높은 fee** 로 교체하라고 권고합니다. MetaMask도 같은 nonce의 self-send/no-op으로 취소·교체를 안내합니다. BitGo는 아예 nonce hole을 별도 API로 탐지·복구하고, Alchemy Wallet APIs는 pending이 너무 길어지면 다시 prepare/send해서 교체하는 흐름을 문서화합니다. 19

메타트랜잭션·멀티시그·스마트계정은 “nonce를 없애는” 것이 아니라 **nonce의 위치와 도메인을 바꿉니다**. OpenZeppelin Relay는 EOA relay의 nonce를 서비스가 관리하고, Safe는 Safe contract nonce에 대해 오프체인 서명을 수집합니다. ERC-4337은 EntryPoint의 `key + sequence` 구조를 통해 하나의 계정 안에서도 여러 nonce domain을 만들 수 있습니다. Alchemy의 `nonceKey` 병렬 전송은 이 발상을 제품화한 예입니다. 20

패턴	장점	단점	적합한 곳	근거
RPC 조회만 사용	구현 단순	병렬 처리 시 충돌·갭 위험	소규모 순차 전송	21
로컬 nonce tracker + mutex	클라이언트/서비스 내 충돌 억제	다중 인스턴스 분산 환경에는 약함	단일 프로세스 지갑	22
중앙 nonce service	다중 워커/노드 간 일관성 우수	SPOF/락 경합 관리 필요	거래소·출금 시스템	23
관리형 relay/custody	키보관·가스·재시도까지 위임 가능	벤더 종속, 내부 알고리즘 불투명	빠른 출시, 기관 운영	24
ERC-4337 keyed nonce	병렬성 우수, batching/paymaster 결합 용이	bundler/paymaster 의존성, 체인별 성숙도 차이	고성능 앱, 가스 리스 UX	25
멀티시그/오프체인 서명 수집	승인 흐름·감사성 우수	서명 집계 지연, 별도 서비스 필요	treasury, DAO, 기관 결재	26
다중 출금 지갑 샤딩	한 지갑 stuck 시 전체 정지 완화	잔액 재분배·키 관리 복잡	대량 출금, 거래소	27
API idempotency 키 매핑	중복 요청/네트워크 재시도 안전	키 TTL·재사용 정책 설계 필요	모든 외부 API 연계	28



실무 선택은 보통 이분법이 아니라 조합입니다. 예를 들어 거래소는 **중앙 nonce service + 다중 출금 지갑 + private relay** 를, 소비자 앱은 **관리형 smart wallet + idempotency + 상태 웹훅** 을 택하는 경우가 많습니다. 29

체인별 차이

체인군	관리해야 할 개념	저장 위치	운영 포인트	근거
EVM	계정 nonce	온체인 account state, 오프체인 pending store	순차 강제, gap이 뒤 거래를 막음. 같은 nonce+더 높은 fee로 대체. chainId로 cross-chain replay 방지	30
Bitcoin	계정 nonce 없음. UTXO/outpoint, nSequence, RBF	UTXO set·mempool	UTXO 예약/coin selection 이 핵심. 교체는 BIP125 RBF로 처리	31
Solana	recent blockhash, durable nonce account	txn message / nonce account	recent blockhash는 150 slots만 유효. 지연 서명은 durable nonce account 필요	32
Algorand	FirstValid/LastValid + Lease	txn header	최대 1000 rounds 유효. 같은 sender+lease는 유효 구간 내 중복 확정 불가	33
Starknet	account contract nonce	계정 계약 상태	revert 후에도 nonce가 증가하고, mempool은 미래 nonce 허용폭과 replacement 상황 규칙이 있음	34
Cosmos SDK	account sequence	계정 상태	sequence mismatch가 대표 오류. 성공 시 sequence 증가	35

따라서 “nonce 서비스”의 키 설계도 체인별로 달라져야 합니다. EVM은 (chainId, sender) 가 기본이고 4337이면 (chainId, sender, nonceKey) 까지, Safe는 (chainId, safeAddress), Solana는 (cluster, nonceAccount), Algorand는 (network, sender, lease) 가 자연스럽습니다. 이는 체인 모델이 다르기 때문에 생기는 필수 분기입니다. 36

운영·보안·확장성

운영·보안 측면에서 nonce는 **키 관리와 분리하되, 제출 권한과는 강하게 묶어야** 합니다. Fireblocks는 MPC·secure enclave/HSM·active/active 서명 컴포넌트를 권고하고, Ready는 backend guardian/co-signer 구조를 공개합니다. Safe Transaction Service는 PostgreSQL·Redis·RabbitMQ·Celery 기반으로 오프체인 서명과 인덱싱을 분리하고, reorg 시 온체인 인덱스만 롤백하되 오프체인 서명은 잃지 않도록 설계했습니다. 37

공격 시나리오도 nonce 설계와 연결됩니다. 리플레이는 EIP-155의 chain ID로 막고, 프론트런/샌드위치는 private mempool로 완화할 수 있습니다. Infura와 Alchemy는 raw transaction을 private infrastructure/virtual mempool로 라우팅하는 MEV 보호를 제공하지만, 이 경우 public mempool 관찰 기반 모니터링은 약해질 수 있어 내부 상태 추적을 더 강화해야 합니다. ³⁸

확장성 측면에서 병목 완화 수단은 세 가지가 검증되었습니다. 첫째, **여러 출금 지갑**으로 샤딩해 단일 nonce head-of-line blocking을 줄이는 것. 둘째, **배치 실행**으로 사용자 의도를 한 번에 묶는 것. 셋째, **4337 keyed nonce**로 병렬 도메인을 늘리는 것입니다. Fireblocks와 Kaia는 다중 계정/지갑 분산을 권하고, Coinbase·Alchemy·Safe는 스마트 계정의 배치·가스 추상화·번들러를 전면에 둡니다. ³⁹

다음 지표와 임계값은 문헌의 실패 모드에 근거한 **초기 운영 권고치**입니다. 실제 수치는 체인별 최종성·SLA·거래량에 맞춰 보정해야 합니다. ⁴⁰

지표	의미	초기 경보 규칙
<code>nonce_gap_count</code>	내부 할당 nonce와 온체인/네트워크 head 사이에 구멍 존재	0 초과 즉시 경보
<code>oldest_pending_age</code>	가장 오래된 미확정 tx 연령	EVM: 최근 P95 확정시간의 2배 초과 시 경보. Solana: 45초 이상 주의, 60초 이상 치명. Algorand: 유효창의 80% 소진 시 경보
<code>replacement_rate</code>	동일 nonce 재전송 비율	15분 이동창에서 10% 초과 시 fee 모델 점검
<code>stuck_tx_count</code>	stuck 상태 tx 수	1건 이상이면 운영자 확인, 3건 이상이면 자동 가스 상향/대체
<code>wallet_liquidity_low</code>	샤딩된 hot wallet 잔액 부족	예상 1시간 출금량 미만이면 refill
<code>rpc_state_divergence</code>	서로 다른 RPC의 pending/receipt 해석 차이	두 공급자 간 상태 불일치가 1블록 이상 지속되면 장애 후보

권장 설계와 검증 과제

권장 설계는 “중앙 nonce service + 관리형 idempotency + 상태 watcher + same-nonce replacement + sender pool 샤딩”입니다. EVM 기준으로는 `(chainId, sender, nonceKey(optional))`를 기본 키로 하고, `eth_getTransactionCount("pending")`는 초기 동기화와 주기적 재조정용으로만 쓰는 것이 안전합니다. 제출 API는 idempotency 키를 강제해 **요청 재시도**와 **nonce 재할당**을 분리해야 합니다. 고성능 구간은 EOA 여러 개로 샤딩하거나, 4337 스마트 계정으로 병렬 nonce domain을 확보하는 편이 낫습니다. ⁴¹

구현 체크리스트	권고
상태 키	<code>chain + sender + domain</code> 으로 설계하고 체인별로 domain 의미를 다르게 둔다
저장소	Redis 단독보다 Redis+DB 이중화 또는 직렬화 가능한 DB를 권장
잠금	sender 단위 lock/CAS를 강제하고, 다중 인스턴스에서 local mutex만 믿지 않는다
제출	idempotency 키와 nonce 할당을 분리 저장한다

구현 체크리스트	권고	
재시도	동일 nonce 교체를 기본 전략으로 하고, gap repair 엔드포인트를 별도로 둔다	
관찰	pending·receipt·reorg·provider divergence·private relay 제출 결과를 모두 기록한다	
보안	HSM/MPC, 정책 엔진, signer 분리, 지갑별 한도/속도 제한을 적용한다	
확장	volume 증가 시 다중 hot wallet, batch 실행, 4337 keyed nonce 중 하나 이상을 도입한다	
오픈 질문 또는 리스크	왜 중요한가	권장 PoC
Solana durable nonce의 장기 방향성	공식 문서가 향후 deprecated 가능성을 언급	offline signing이 필요한 흐름을 recent blockhash 재빌드 방식과 비교 시험
private mempool 사용 시 관찰성 저하	Infura·Alchemy는 public mempool 밖으로 라우팅	public/MEV-protected 경로 각각에서 status 지연·replacement 성공률 비교
4337 병렬 nonce의 실효성	keyed nonce는 이론상 강력하지만 bundler/paymaster 행태 차이 존재	동일 워크로드를 EOA 단일 nonce vs 4337 keyed nonce로 부하 시험
벤더별 내부 nonce 알고리즘 불투명성	Binance/Argent/국내 거래소는 공개 수준이 제한적	provider 변경·장애·중복 제출 시 응답과 실제 온체인 상태 차이를 회귀 테스트
hole repair 자동화의 안전성	자동 복구가 잘못되면 중복 송금 또는 취소 오판 가능	BitGo 스타일 potentialStuckTx 탐지 후 noop/cancel/replace 세 경우를 시뮬레이션

결론적으로, “**nonce를 온체인에서 읽는다**”는 것은 출발점일 뿐이고, 실제 서비스는 “**오프체인에서 의도를 직렬화하고 온체인과 화해시키는 문제**”입니다. 공개 사례 기준 최선의 기본값은 EVM에서 중앙 nonce service를 두고, 다중 송신자 샤딩과 private relay 및 idempotency를 결합하는 것입니다. 스마트 계정이 가능한 제품에서는 4337 keyed nonce와 batching이 가장 유망한 병목 완화 수단입니다. 반대로 Bitcoin·Solana·Algorand처럼 nonce가 없거나 다른 형태인 체인은 **동일한 EVM nonce 엔진을 이식하지 말고**, UTXO lock, blockhash lifecycle, validity window/lease를 각각 별도 모듈로 다뤄야 합니다. ⁴²

¹ ⁴ ¹⁸ ²² GitHub - MetaMask/nonce-tracker · GitHub

<https://github.com/MetaMask/nonce-tracker>

² Send parallel transactions | Alchemy Docs

<https://www.alchemy.com/docs/wallets/transactions/send-parallel-transactions>

³ Bitcoin Core :: Opt-in RBF FAQ

https://bitcoincore.org/en/faq/optin_rbf/

⁵ Send Transactions - Coinbase Developer Documentation

<https://docs.cdp.coinbase.com/wallets/using-wallets/send-transactions>

⁶ What is a Nonce in the EVM-Compatible Blockchains? | Binance Nonce, Binance EVM, What is a Nonce

<https://www.binance.com/bg/support/faq/detail/97def86be5f34602a23eee310c65a13a>

⁷ ¹⁶ ²¹ Ethereum eth_getTransactionCount | MetaMask developer documentation

https://docs.metamask.io/services/reference/ethereum/json-rpc-methods/eth_gettransactioncount/

- 8 **Retry Transactions | Alchemy Docs**
<https://www.alchemy.com/docs/wallets/transactions/retry-transactions>
- 9 **safe-smart-account/contracts/Safe.sol at main**
https://github.com/safe-global/safe-contracts/blob/main/contracts/Safe.sol?utm_source=chatgpt.com
- 10 **Key Management**
https://docs.ready.co/ready-wallets/web-wallet/key-management?utm_source=chatgpt.com
- 11 24 **Manage Withdrawals at Scale - Fireblocks Developer Docs**
<https://developers.fireblocks.com/docs/manage-withdrawals-at-scale>
- 12 **BitGo Wallet Types**
<https://developers.bitgo.com/docs/wallet-types>
- 13 17 19 23 40 41 42 **How to Manage Nonces for Reliable Transactions | Kaia Docs**
<https://docs.kaia.io/build/cookbooks/how-to-manage-nonce/>
- 14 **Get Next Nonce by Account | Nodit Documentation**
<https://developer.nodit.io/en/api/ethereum/web3-data-api/getnextnoncebyaccount/>
- 15 **Open API 정식 오픈 안내 및 사용 가이드**
<https://docs.upbit.com/kr/changelog/%EC%95%88%EB%82%B4-open-api-%EC%A0%95%EC%8B%9D-%EC%98%A4%ED%94%88-%EC%95%88%EB%82%B4-%EB%B0%8F-%EC%82%AC%EC%9A%A9-%EA%B0%80%EC%9D%B4%EB%93%9C>
- 20 **Relaying gasless meta-transactions with a web app | OpenZeppelin Docs**
<https://docs.openzeppelin.com/defender/guide/meta-tx>
- 25 36 **account-abstraction/contracts/core/NonceManager.sol at develop · eth-infinitism/account-abstraction · GitHub**
<https://github.com/eth-infinitism/account-abstraction/blob/develop/contracts/core/NonceManager.sol>
- 26 **Running the Safe Transaction Service – Safe Docs**
<https://docs.safe.global/core-api/api-safe-transaction-service>
- 27 29 37 39 **Creating an efficient and robust withdrawal system for crypto assets | Fireblocks**
<https://www.fireblocks.com/blog/creating-an-efficient-and-robust-blockchain-withdrawal-system>
- 28 **Idempotency - Coinbase Developer Documentation**
https://docs.cdp.coinbase.com/api-reference/v2/idempotency?utm_source=chatgpt.com
- 30 **Ethereum accounts**
https://ethereum.org/developers/docs/accounts/?utm_source=chatgpt.com
- 31 **Transactions**
https://developer.bitcoin.org/reference/transactions.html?utm_source=chatgpt.com
- 32 **Transactions**
https://solana.com/docs/core/transactions?utm_source=chatgpt.com
- 33 **Leases | Algorand Developer Portal**
<https://dev.algorand.co/concepts/transactions/leases/>
- 34 **Accounts - Starknet Documentation**
<https://docs.starknet.io/learn/protocol/accounts>
- 35 **Accounts - Cosmos Docs**
<https://cosmos-docs.mintlify.app/sdk/latest/learn/concepts/accounts>

38 EIP-155: Simple replay attack protection

https://eips.ethereum.org/EIPS/eip-155?utm_source=chatgpt.com